LABORATOR 1 – statistica

EX#1 (**Paradoxul lui Bertrand**) Fie două cercuri concentrice de rază $R = 1$, respectiv $r = R/2$. Vrem să estimăm numeric (frecvenţionist) probabilitatea p ca o coardă generată aleator pe cercul de rază R să intersecteze cercul de rază r. Creaţi un fişier în Python® prin care:

(a) să se genereze aleator o coardă pe cercul de rază R prin generarea aleatoare uniformă a capetelor coardei P_1, P_2 pe cercul de rază R (i.e. $P_1 = (R\cos(\theta_1), R\sin(\theta_1)), P_2 = (R\cos(\theta_2), R\sin(\theta_2))$, unde unghiurile θ_1, θ_2 sunt generate aleator uniform în $[0, 2\pi)$);

(b) să se genereze aleator o coardă pe cercul de rază R prin generarea aleatoare a mijlocului coardei M astfel: $M = (\tilde{r}\cos\theta, \tilde{r}\sin\theta)$, unde raza $\tilde{r}$ este generată aleator uniform în $(0, R)$, iar unghiul θ este generat aleator uniform în $[0, 2\pi)$);

(c) să se genereze aleator o coardă pe cercul de rază R prin generarea aleatoare uniformă în discul de rază R a mijlocului coardei M (i.e. $M = (\sqrt{\tilde{r}}\cos\theta, \sqrt{\tilde{r}}\sin\theta)$, unde $\tilde{r}$ este generat aleator uniform în $(0, R^2)$, iar unghiul θ este generat aleator uniform în $[0, 2\pi)$);

(d) să se estimeze numeric (frecvenţionist) probabilitatea p pentru fiecare dintre cele trei metode de generare aleatoare a coardei de la (a), (b), respectiv (c);

(e) să se reprezinte grafic într-un sistem xOy coardele generate la (d) împreună cu mijloacele lor, pentru fiecare dintre cele trei metode de generare de la (a), (b), respectiv (c).

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
# ----------------------------
# Metoda (a): capete uniforme pe cerc
# ----------------------------
N = int(1e7)    # poţi pune 1e7 dar va merge mai lent


# 1) Generăm unghiurile (codul profei)
theta = 2*np.pi*np.random.rand(N, 2)
# np.random.rand(N, 2) - creeaza o matrice c N linii si 2 coloane
#fiecare element este un număr aleator uniform în intervalul [0, 1)
# o linie = o coarda
# inmultim cu 2pi, deci intervalul devine [0,2pi]
```

```
# 2) Convertim în coordonate pe cercul de rază 1
X = np.cos(theta)
Y = np.sin(theta)

#P1 = (X[i,0], Y[i,0])
#P2 = (X[i,1], Y[i,1])

# 3) Mijlocul coardei (codul profei)
M = 0*X
#X are aceeaşi formă ca theta → (N, 2)
#0*X creează o matrice ZERO cu aceeaşi formă ca X
M[:,0] = (X[:,0] + X[:,1]) / 2
# M[:,0] - ia toate randurile de pe coloana 0
M[:,1] = (Y[:,0] + Y[:,1]) / 2
# M[:,1] - ia toate randurile de pe coloana 1

# 4) Distanţa la origine
lm = np.linalg.norm(M, axis=1)
#axis=1 spune: aplică norma pe fiecare rând (nu pe întreaga matrice)

# 5) Numărăm cazurile favorabile
ncazfav = np.sum(lm <= 0.5)
P = ncazfav / N
print("Probabilitatea estimată (metoda a) =", P)
```

```
# ----------------------------
# Plot (codul tău inițial)
# ----------------------------
plt.figure() # Creează o nouă figură pentru plot.
plt.xlim([-1,1]) #Setează limitele axe X (orizontală) între -1 și 1.
plt.ylim([-1,1]) #Setează limitele axe Y (verticală) între -1 și 1.
plt.axis('equal') # equal' → păstrează proporțiile X și Y egale
plt.title('Method 1: Uniform endpoints on circle')

circle = Circle((0, 0), 1, edgecolor='grey', facecolor='none')
# Aceasta creează un obiect cerc în matplotlib
#(0, 0) → centru cercului
#1 → raza cercului
#edgecolor='grey' → culoarea conturului cercului
#facecolor='none' → cercul nu este umplut (transparent în interior)
circle12 = Circle((0, 0), 1/2, edgecolor='grey', facecolor='none')
plt.gca().add_patch(circle)
plt.gca().add_patch(circle12)
#gca = get current axes → "ia axele curente ale plotului"
#Practic, asta ne permite să adăugăm obiecte pe grafic, cum sunt cercurile
#add_patch → adaugă un patch (obiect grafic) în plot
#circle este obiectul cerc pe care l-am creat mai devreme

#Rezultat: cercul apare în grafic
# desenăm doar 200 de coarde ca să nu încărcăm graficul
for i in range(10):
```

```
    plt.plot([X[i,0], X[i,1]],
             [Y[i,0], Y[i,1]],
             color='green', alpha=0.3)
    plt.scatter(M[i,0], M[i,1], color='red', s=5)

plt.show()
```

--b

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle

# ----------------------------
# Metoda (b): mijlocul aleator în R
# ----------------------------

N = int(1e7)   # număr coarde

R = 1          # raza cercului mare
r_small = 0.5  # raza cercului mic

# 1) Generăm M aleator în cercul mare
theta = 2*np.pi*np.random.rand(N)       # unghi uniform [0, 2pi)
r = R*np.random.rand(N)                 # raza uniform [0, R)
#distanţa punctului M faţă de centru (0...R)
```

```
#np.random.rand(N) → valori uniforme între 0 şi 1
#Înmulţim cu R → valori uniforme între 0 şi R (raza cercului)
#Aceasta defineşte cât de departe de centru va fi punctul M
# coordonate x şi y ale mijlocului

M = np.zeros((N,2))
M[:,0] = r * np.cos(theta)
M[:,1] = r * np.sin(theta)

# 2) Lungimea OM
lm = np.linalg.norm(M, axis=1)

# 3) Coarde favorabile (intersectează cercul mic)
ncazfav = np.sum(lm <= r_small)
P = ncazfav / N
print("Probabilitatea estimată (metoda b) =", P)

# ----------------------------
# Plot
# ----------------------------
plt.figure()
plt.xlim([-1,1])
plt.ylim([-1,1])
plt.axis('equal')
plt.title('Method 2: Midpoint random in radius')
```

```
circle = Circle((0, 0), R, edgecolor='grey', facecolor='none')
circle12 = Circle((0, 0), r_small, edgecolor='grey', facecolor='none')
plt.gca().add_patch(circle)
plt.gca().add_patch(circle12)

# Desenăm 200 coarde pentru vizualizare
for i in range(200):
    # Generăm capetele coardei pornind de la mijloc şi raza la marginea cercului
    # Dacă ştim mijlocul M şi raza cercului, unghiul perpendicular produce capetele
    # Simplificăm: generăm capetele pe cerc la o direcţie aleatoare
   # alpha = np.random.rand() * 2*np.pi
   # d = np.sqrt(R**2 - lm[i]**2)          # jumătate lungimea coardei
   # dx = d * np.cos(alpha + np.pi/2)
   # dy = d * np.sin(alpha + np.pi/2)
   delta = 0.05  # lungime segment
   plt.plot([M[i,0]-delta, M[i,0]+delta],
        [M[i,1]-delta, M[i,1]+delta],
        color='blue', alpha=0.3)
   plt.scatter(M[i,0], M[i,1], color='red', s=5)

plt.show()
```

--c

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle

# ----------------------------
# Metoda (c): mijlocul uniform în disc
# ----------------------------

N = int(5e5)   # număr coarde

R = 1          # raza cercului mare
r_small = 0.5  # raza cercului mic

# 1) Generăm M aleator uniform în disc
theta = 2*np.pi*np.random.rand(N)       # unghi uniform [0, 2pi)
r_tilde = R**2 * np.random.rand(N)      # r̃ uniform în [0, R^2]

r = np.sqrt(r_tilde)                    # transformare pentru uniformitate pe arie
M = np.zeros((N,2))
M[:,0] = r * np.cos(theta)
M[:,1] = r * np.sin(theta)

# 2) Lungimea OM
lm = np.linalg.norm(M, axis=1)
```

```
# 3) Coarde favorabile (intersectează cercul mic)
ncazfav = np.sum(lm <= r_small)
P = ncazfav / N
print("Probabilitatea estimată (metoda c) =", P)

# ----------------------------
# Plot
# ----------------------------
plt.figure()
plt.xlim([-1,1])
plt.ylim([-1,1])
plt.axis('equal')
plt.title('Method 3: Midpoint uniform in disc')

circle = Circle((0, 0), R, edgecolor='grey', facecolor='none')
circle12 = Circle((0, 0), r_small, edgecolor='grey', facecolor='none')
plt.gca().add_patch(circle)
plt.gca().add_patch(circle12)

# Desenăm 200 coarde pentru vizualizare
for i in range(200):
    #alpha = np.random.rand() * 2*np.pi
    #d = np.sqrt(R**2 - lm[i]**2)
    #dx = d * np.cos(alpha + np.pi/2)
   # dy = d * np.sin(alpha + np.pi/2)
```

```
    plt.plot([M[i,0]-0.05, M[i,0]+0.05],
             [M[i,1]-0.05, M[i,1]+0.05],
             color='purple', alpha=0.3)
    plt.scatter(M[i,0], M[i,1], color='red', s=5)

plt.show()
```

EX#2 (**Paradoxul zilei de naştere**) Vrem să estimăm numeric (frecvenţionist) probabilitatea p ca dintr-un grup de $n \geq 2$ persoane, cel puţin două persoane să aibă aceeaşi zi de naştere. Presupunem că sunt posibile 365 zile de naştere şi că este la fel de probabil ca ziua de naştere a fiecărei persoane din grup să fie oricare dintre aceste zile, independent de celelalte persoane din grup. Creaţi un fişier în Python® prin care:

(a) să se estimeze numeric (frecvenţionist) probabilitatea ca dintr-un grup de 23 persoane, cel puţin două persoane să aibă aceeaşi zi de naştere;

(b) să se estimeze numeric (frecvenţionist) probabilitatea ca dintr-un grup de $n \in \{22, 70, 183\}$ persoane, cel puţin două persoane să aibă aceeaşi zi de naştere.

Indicaţii Python®: `numpy`, `numpy.random.rand`, `matplotlib`, `matplotlib.pyplot`


```
--pb2
import numpy as np
# n = nr persoane din grup
# Nsim = nr de simulari
# P = probabilitate

def birthday_paradox(n, Nsim=int(10000)):
    count = 0
    birthdays = np.random.randint(1,366,size=(Nsim,n))
```

```
    #size=(Nsim, n) → forma array-ului generat este Nsim rânduri × n coloane
    #Fiecare rând reprezintă o simulare a unui grup de n persoane
    #Fiecare coloană reprezintă ziua de naştere a unei persoane din grup
    has_duplicate = np.array([len(np.unique(row)) < n for row in birthdays])
    p = np.mean(has_duplicate) #media aritmetica a elementelor din array
    #print(p)
    return p




def birthday_paradox2(n, Nsim=100000):
    """
    n     : numărul de persoane în grup
    Nsim   : numărul de simulări Monte Carlo
    return : probabilitatea estimată ca cel puţin două persoane să aibă aceeaşi zi de naştere
    """
    count = 0
    for _ in range(Nsim):
        # Generăm n zile de naştere aleator
        birthdays = np.random.randint(1, 366, size=n)
        # Verificăm dacă există duplicate
        if len(np.unique(birthdays)) < n:
            count += 1
    return count / Nsim
```

```
num_people = 23
P = birthday_paradox2(num_people)
print(P)

num_people = 70
P = birthday_paradox(num_people)
print(P)
```

MAI RAPID:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
# ----------------------------
# Metoda (a): capete uniforme pe cerc
# ----------------------------
N = int(1e7)

theta = 2 * np.pi * np.random.rand(N,2)
```

```
X = np.cos(theta)
Y = np.sin(theta)

M = np.zeros_like(X)
M[:,0] = (X[:,0] + X[:,1]) / 2
M[:,1] = (Y[:,0] + Y[:,1]) / 2

lm = np.linalg.norm(M, axis = 1)

caz_fav = np.sum(lm <= 0.5)
P = caz_fav/N
print(P)

# b
N = int(1e7)   # număr coarde

R = 1          # raza cercului mare
r_small = 0.5  # raza cercului mic

# 1) Generăm M aleator în cercul mare
theta = 2*np.pi*np.random.rand(N)      # unghi uniform [0, 2pi)
r = R*np.random.rand(N)                # raza uniform [0, R)
#distanța punctului M față de centru (0...R)

#np.random.rand(N) → valori uniforme între 0 și 1
#Înmulțim cu R → valori uniforme între 0 și R (raza cercului)
```

```
#Aceasta definește cât de departe de centru va fi punctul M
# coordonate x și y ale mijlocului

M = np.zeros((N,2))
M[:,0] = r * np.cos(theta)
M[:,1] = r * np.sin(theta)

# 2) Lungimea OM
lm = np.linalg.norm(M, axis=1)

# 3) Coarde favorabile (intersectează cercul mic)
ncazfav = np.sum(lm <= r_small)
P = ncazfav / N
print("Probabilitatea estimată (metoda b) =", P)

#c
R = 1
r_small = 0.5

r = R**2 * np.random.rand(N)
theta= 2 * np.pi * np.random.rand()

coef = np.sqrt(r)

M[:,0] = coef * np.cos(theta)
M[:,1] = coef * np.sin(theta)
```

```
lm = np.linalg.norm(M, axis = 1)

nr_fav = np.sum(lm <= 0.5)

P = nr_fav / N
print("Probabilitatea estimată (metoda c) =", P)


plt.figure()
plt.axis('equal')
plt.xlim(-1,1)
plt.ylim(-1,1)

circle1 = Circle((0,0), 1, edgecolor = 'green', facecolor = 'none')
circle2 = Circle((0,0), 1/2, edgecolor = 'green', facecolor = 'none')

plt.gca().add_patch(circle1)
plt.gca().add_patch(circle2)

for i in range(20):
    plt.plot([X[i,0], X[i,1]], [Y[i,0], Y[i,1]])
```

```
    plt.scatter(M[i,0], M[i,1])


plt.show()
```

```
#ex 2
def zile_nastere(n, N = 10000):
    random_try = np.random.randint(1,366, size = (N, n))
    unique_elem = np.array([len(np.unique(rand)) < n for rand in
random_try])
    p = np.mean(unique_elem)
    return p

print("23: ", zile_nastere(23))
print("22: ", zile_nastere(22))
print("70: ", zile_nastere(70))
print("183: ", zile_nastere(183))
```